

PROJECT ZENITH

The Celestial Eye

A real-time celestial & satellite tracking platform built around the **Zenith Window** — a topocentric radar cone that surfaces every object passing nearly directly overhead.

PROJECT

TEAM

EVENT

CORE INNOVATION

STACK

DOCUMENT

GENERATED

Zenith — The Celestial Eye

Cipher (Aryan · Ashraf Khan)

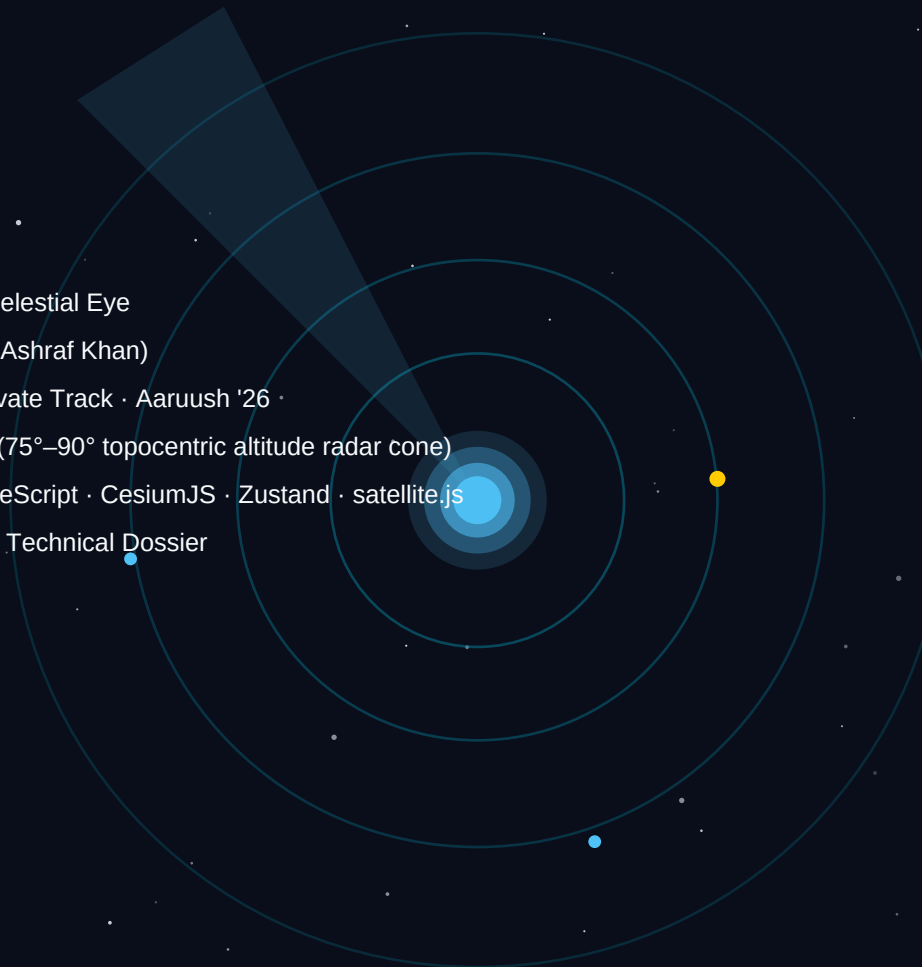
AstralWeb Innovate Track · Aaruush '26

Zenith Window (75°–90° topocentric altitude radar cone)

Next.js 14 · TypeScript · CesiumJS · Zustand · satellite.js

Comprehensive Technical Dossier

25 June 2026



Contents

1. Executive Summary

1.1 What Makes It Distinctive

55

2. Technology Stack & Architecture

2.1 Core Technologies

2.2 External Data Sources

2.3 Project Structure

2.4 High-Level Data Flow

66777

3. The Zenith Window — Core Concept & Mathematics

3.1 What "Topocentric Altitude" Means

3.2 The Coordinate Transformation Pipeline

3.3 The Zenith Score (0–100)

8899

4. The Live Data Pipeline

4.1 The Refresh Loop

4.2 Off-Thread Propagation (Web Worker)

4.3 Caching & Rate-Limit Strategy

10101010

5. State Management — The Zustand Store

5.1 Store Shape

5.2 Key Actions

111111

6. 3D Rendering — The CesiumJS Layer

6.1 Scene Configuration

6.2 The Zenith Cone

6.3 Object Markers & Styling Rules

6.4 Orbital Trail Lines

1313131314

7. Complete Feature Catalogue	15
7.1 Observer Location Picker	15
7.2 Pass Predictions	15
7.3 Object Detail Panel	15
7.4 Time Machine	16
7.5 Zenith Radar Minimap	16
7.6 Third-Person Satellite Chase View	16
7.7 Solar-System View	16
7.8 Cinematic Camera Choreography	17
7.9 Constellation View	18
8. UI / UX Design System	19
8.1 Colour Language	19
8.2 Glassmorphism Recipe	19
8.3 Typography	19
8.4 Motion & Animation Inventory	19
8.5 The Decluttering Principle	20
9. Server-Side API Routes	21
9.1 ISS — Dual-Source Promotion	21
9.2 Planets — Horizons Ephemeris	21
10. Type System	22
11. Performance Engineering	23
11.1 The Diagnosis	23
11.2 The Six Fixes	23
12. Engineering Lessons & Key Principles	25
12.1 Technical Gotchas (Hard-Won)	25

12.2 Workflow Principles 25

13. Build Timeline 26

13.1 Current State 26

Project Zenith — The Celestial Eye 26

1. Executive Summary

Project Zenith — The Celestial Eye is a real-time, browser-based celestial tracking application that reframes satellite tracking as an act of *looking up*. Where conventional trackers present a flat, scrolling list of orbital elements, Zenith renders a living 3D Earth and projects a translucent **radar cone** straight up from the observer's location. Any satellite, the ISS, or a planet that drifts into the cone — specifically into the **75° to 90° topocentric altitude band**, i.e. nearly directly overhead — is highlighted, named, and surfaced into a curated dashboard.

The application tracks roughly **750 live satellites** propagated from real Two-Line Element sets, overlays the live ISS position, places the visible planets via NASA ephemerides, predicts upcoming overhead passes, lets the user fly a cinematic third-person chase camera behind any satellite, zoom out into a fully orbiting **solar-system view**, and reveal the **constellations** currently overhead. All of this runs at an interactive frame rate through an aggressively optimized CesiumJS rendering pipeline.

THE ONE-SENTENCE PITCH

Zenith turns 750+ orbiting objects and the night sky into a single, beautiful, real-time answer to the question: "**what is above me right now?**"

1.1 What Makes It Distinctive

- **The Zenith Window concept.** The 75°–90° altitude cone is the product's signature. It converts an abstract catalogue into a spatial, glanceable experience and is the conceptual hook the entire UI is organized around.
- **Real data, end to end.** Orbital positions come from live CelesTrak TLEs propagated with the industry-standard SGP4 model; the ISS position is cross-checked against OpenNotify's NASA-fed feed; planets come from NASA/JPL Horizons.
- **Judged on UI, engineered for UI.** A dark-space glassmorphism design system, an animated Zenith Score, a sonar-pulsing radar minimap, cinematic camera flights, and breathing-cone animation are all built to make the interface feel alive.
- **Performance as a feature.** The render pipeline was rebuilt from per-object Cesium entities to batched primitive collections with render-on-demand, taking the app from a stuttering swarm to a smooth, idle-when-static experience.

2. Technology Stack & Architecture

Zenith is a single Next.js application. The browser renders the 3D globe and all UI; Next.js server-side API route handlers act as a proxy layer in front of every external data source, which keeps third-party API keys server-side and side-steps the CORS restrictions that CelesTrak, Horizons, and Nominatim impose on direct browser requests. Heavy orbital mathematics is pushed off the main thread into a Web Worker so the render loop never blocks.

2.1 Core Technologies

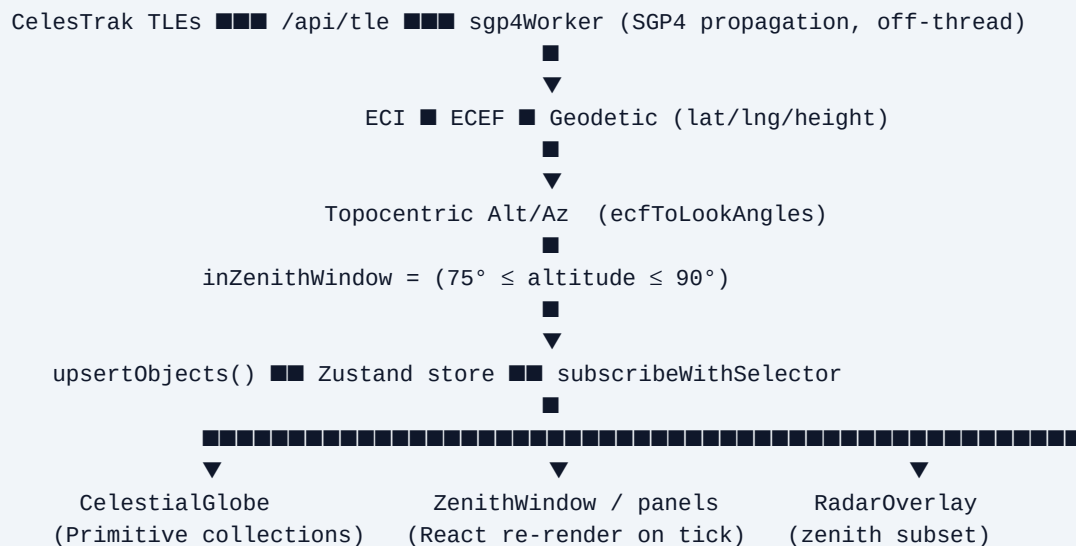
Layer	Technology	Role in Zenith
Framework	Next.js 14 (App Router)	Pages, layout, and server-side API route handlers
Language	TypeScript	End-to-end type safety across pipeline, store, and UI
Styling	Tailwind CSS	Utility-first glassmorphism design system
3D Engine	CesiumJS	WebGL globe, entities, primitive collections, camera
Globe Imagery	Cesium Ion	Bing Maps Aerial (asset 2), terrain, lighting, atmosphere
State	Zustand	Central store with subscribeWithSelector middleware
Orbital Math	satellite.js	SGP4/SDP4 propagation from TLE data
Concurrency	Web Worker	Off-thread SGP4 propagation + pass prediction
Typography	Space Grotesk + Space Mono	Display headings + monospace telemetry

2.2 External Data Sources

Source	Endpoint	Provides
CelesTrak	GP endpoint (gp.php?GROUP=active&FORMAT=json)	Two-Line Element sets for ~750 active satellites
OpenNotify	iss-now.json	Live ISS latitude / longitude / altitude (NASA-fed)
NASA/JPL Horizons	horizons.api (OBSERVER ephemeris)	Geocentric RA/Dec + range for the visible planets
OpenStreetMap Nominatim	search?q=&format=json	City-name → latitude/longitude geocoding for the observer picker

WHY EVERYTHING IS PROXIED THROUGH /app/api/

CelesTrak, Horizons, and Nominatim all reject cross-origin browser requests or require descriptive User-Agent headers. Routing each one through a Next.js route handler means the browser only ever talks to our own origin, the Cesium Ion token stays out of client bundles, and each source gets its own caching + graceful-failure policy in one place.



3. The Zenith Window — Core Concept & Mathematics

The Zenith Window is the intellectual centre of the project. This section explains what it means, why the 75°–90° band was chosen, and the full chain of coordinate transformations that turns a raw orbital element set into the single boolean `inZenithWindow` that drives the entire visual experience.

3.1 What "Topocentric Altitude" Means

Altitude (in the astronomical sense, also called elevation) is the angle of an object above the observer's local horizon. 0° sits on the horizon; 90° is the **zenith** — the point directly overhead. The word **topocentric** means the angle is measured from the observer's actual position on the Earth's surface, not from the Earth's centre. An object at 88° altitude is, from the observer's point of view, almost exactly overhead.

Zenith defines its window as $75^\circ \leq \text{altitude} \leq 90^\circ$. Anything inside that 15° cap of sky is, loosely speaking, "above you right now." This is the set the radar cone visualizes and the dashboard curates.

THE GOVERNING CONSTANT

```
export const ZENITH_WINDOW = { minAlt: 75, maxAlt: 90 }  
inZenithWindow = topo.altitude >= 75 && topo.altitude <= 90
```

A single source of truth, imported everywhere the window is referenced.

3.2 The Coordinate Transformation Pipeline

Going from a TLE to an Alt/Az pair is a seven-step chain. Each step lives in the `/lib` pipeline and runs inside the worker for every object, every tick.

1. Fetch TLE

A Two-Line Element set is retrieved from CelesTrak. It encodes the satellite's orbit as a compact set of mean orbital elements at a reference epoch.

2. SGP4 Propagation

`satellite.js's sgp4()` propagates the TLE to the current timestamp, producing a position vector in the **ECI** (Earth-Centred Inertial) frame — a frame fixed relative to the stars.

3. ECI → ECEF

The ECI vector is rotated into the **ECEF** (Earth-Centred, Earth-Fixed) frame using the current **GMST** (Greenwich Mean Sidereal Time), so the coordinate frame now rotates with the Earth.

4. ECEF → Geodetic

`eciToGeodetic()` converts the position to **geodetic** latitude, longitude, and height on the WGS-84 ellipsoid — the form CesiumJS needs to place the dot on the globe.

5. Topocentric Alt/Az

`ecfToLookAngles()` takes the observer's location and the satellite's ECEF position and returns the look angles: azimuth, elevation (altitude), and range.

6. Window Test

If the resulting altitude lies in $[75^\circ, 90^\circ]$, the object's `inZenithWindow` flag is set true.

7. Store Upsert

`upsertObjects()` merges the fresh positions into the Zustand store, which auto-recomputes the filtered `zenithObjects` array and notifies every subscriber.

■ THE RADIANS BUG — A HARD-WON LESSON

`ecfToLookAngles()` returns elevation in **radians**, not degrees. Comparing a radian value (≈ 1.5 at the zenith) directly against the $75\text{--}90^\circ$ bounds silently breaks the entire window — nothing ever qualifies. The fix is to convert first: `elevationDeg = elevationRad × (180 / Math.PI)`. This single units mismatch was the most consequential bug of the build and is now documented in the project's persistent context so it can never regress.

3.3 The Zenith Score (0–100)

To gamify "how good is the sky overhead right now," Zenith derives a single **Zenith Score** from the live set of in-window objects. It is the average altitude of everything currently in the Zenith Window, rescaled so that the bottom of the window (75°) maps toward 0 and the zenith (90°) maps toward 100:

```
const avgAlt = zenithObjects.length > 0
  ? zenithObjects.reduce((s, o) => s + o.topocentricPosition.altitude, 0)
    / zenithObjects.length
  : 0;
const score = Math.min(100, Math.round((avgAlt - 75) * 6.67));
// 75° → 0 · 90° → 100 (15° span × 6.67 ≈ 100)
```

The score feeds a circular SVG ring widget whose stroke arc animates to the value over one second, with an animated waveform of oscillating bars beside it to give the impression of a live signal being measured before the number settles.

4. The Live Data Pipeline

The pipeline is the heartbeat of the application: a self-healing, ten-second loop that keeps roughly 750 objects current while degrading gracefully whenever an upstream source falters.

4.1 The Refresh Loop

`lib/refreshLoop.ts` exposes `startRefreshLoop(store)` and a matching `stopRefreshLoop()` cleanup. On a `setInterval` of 10,000 ms it reads the current observer from `store.getState()`, asks the worker to propagate the catalogue against that observer and the current timestamp (plus any Time-Machine offset), then calls `upsertObjects()` with the result. A failed tick is logged and skipped — the loop never throws and never stops. The component starts the loop on mount and tears it down on unmount inside a `useEffect`.

WHY THE OBSERVER IS READ FRESH EACH TICK

`setObserver()` deliberately does **not** recompute topocentric angles itself — doing so against stale satellite positions would be wrong. Instead the next pipeline tick is treated as the single authority: it reads the new observer and re-derives every Alt/Az and the entire zenith set together, consistently.

4.2 Off-Thread Propagation (Web Worker)

SGP4 propagation of ~750 satellites is non-trivial arithmetic. Running it on the main thread would compete with the render loop and cause visible stutter. `lib/sgp4Worker.ts` therefore performs all propagation in a Web Worker. The worker also handles a second message type, `PREDICT_PASSES`, used by the pass-prediction engine. Keeping both heavy workloads off-thread is a primary reason the UI stays responsive.

4.3 Caching & Rate-Limit Strategy

CelesTrak rate-limits aggressively. During development, rapid page reloads repeatedly tripped its limiter and returned connection failures. The project hardens against this on several fronts:

- **Stable endpoint.** The pipeline uses the stable GP JSON endpoint (`gp.php?GROUP=active&FORMAT=json`) rather than the rate-sensitive legacy text feeds.
- **localStorage TLE cache.** Fetched TLEs are cached client-side for two hours, so ordinary reloads never re-hit CelesTrak.
- **Graceful 502.** When every CelesTrak feed fails with no cache available, the `/api/tle` route returns an honest HTTP 502 rather than fabricating data. The TopBar surfaces a  error pill and the loop simply retries on the next tick — the app auto-recovers the moment CelesTrak responds again.
- **Horizons last-known cache.** The planets route keeps the last successful ephemeris in a module-level cache and serves it if Horizons is unreachable.

A DELIBERATE UX DECISION: PLANETS PERSIST ON FAILURE

Because `upsertObjects()` *merges* rather than deletes, a failed planets tick leaves the planets at their last-known positions instead of making them blink out of existence. The result is a flicker-free display that refreshes silently on the next successful fetch — chosen intentionally over momentary disappearance.

5. State Management — The Zustand Store

A single Zustand store, `store/zenithStore.ts`, is the spine the whole application hangs from. It uses the `subscribeWithSelector` middleware so that imperative code (most importantly the CesiumJS layer) can subscribe to a single slice of state and react only when that slice changes — without forcing React re-renders.

5.1 Store Shape

State Field	Type	Purpose
<code>observer</code>	<code>ObserverLocation</code>	Current observer lat/lng/altitude/label
<code>objects</code>	<code>Map<id, CelestialObject></code>	The full live catalogue (~750 objects)
<code>zenithObjects</code>	<code>CelestialObject[]</code>	Derived: only objects with <code>inZenithWindow=true</code>
<code>showZenithCone</code>	<code>boolean</code>	Cone visibility toggle
<code>selectedObjectId</code>	<code>string null</code>	Which object's detail panel is open
<code>trackingObjectId</code>	<code>string null</code>	Which object the chase camera is following
<code>offsetHours</code>	<code>number</code>	Time-Machine offset applied to propagation time
<code>solarSystemVisible</code>	<code>boolean</code>	Whether the solar-system overlay is active
<code>constellationsVisible</code>	<code>boolean</code>	Whether constellations are being drawn
<code>selectedConstellationId</code>	<code>string null</code>	Which constellation is in immersive view

5.2 Key Actions

Action	Effect
<code>upsertObjects(objs[])</code>	Bulk-merges new positions; auto-recomputes <code>zenithObjects</code>
<code>setObserver(observer)</code>	Updates the observer; next tick re-derives all Alt/Az
<code>toggleZenithCone()</code>	Shows / hides the radar cone
<code>setSelectedObjectId(id)</code>	Opens or closes the object detail panel
<code>setTrackingObjectId(id)</code>	Enters / exits third-person chase camera
<code>setOffsetHours(h)</code>	Sets the Time-Machine offset (0 / +1 / +6 / +24 h)
<code>setSolarSystemVisible(v)</code>	Toggles the orbiting solar-system view
<code>setConstellationsVisible(v)</code>	Toggles constellation rendering
<code>setSelectedConstellationId(id)</code>	Enters / exits immersive constellation view

THE CRITICAL ARCHITECTURAL SEAM

CelestialGlobe never consumes the store through a React hook in its render body. It reads state via `useZenithStore.getState()` and subscribes imperatively inside `useEffect`. The consequence is that the globe does **not** React-re-render when 750 positions update every ten seconds — only the lightweight panels do. This separation is fundamental to the app's performance.

6. 3D Rendering — The CesiumJS Layer

`components/CelestialGlobe.tsx` owns everything drawn in 3D. Because CesiumJS is a WebGL engine with no server-side rendering support, the component is always loaded with `dynamic(() => import(...), { ssr: false })`. The Cesium Ion token lives in `.env.local` and the base URL for Cesium's static assets is injected at build time.

6.1 Scene Configuration

The viewer is configured for a cinematic, mission-control aesthetic:

- **Bing Maps Aerial** imagery via Cesium Ion (asset 2) for photoreal terrain.
- **Day/night lighting** — `globe.enableLighting = true` with real sun position, producing a live terminator across the Earth.
- **Atmospheric scattering** — sky atmosphere, fog, and an elevated `atmosphereLightIntensity` for the soft blue limb glow.
- **Bloom post-processing** for a subtle glow on bright points and the sun.
- **Render-on-demand** — `requestRenderMode = true` so the GPU idles between data ticks (covered in depth in §11).

6.2 The Zenith Cone

The cone is a Cesium `CylinderGraphics` entity positioned at the observer's surface point and oriented along the local **up** axis using `eastNorthUpToFixedFrame`, so it always points radially outward, normal to the surface. It is filled with a highly transparent cyan gradient and outlined in a brighter cyan. The cone re-renders whenever the observer changes and is shown or hidden by `showZenithCone`.

To make the cone feel alive it **breathes**: its material colour alpha is driven by a Cesium `CallbackProperty` whose value oscillates on a sine wave each frame, gently pulsing the cone's opacity and outline. A lightweight `requestRender` heartbeat keeps the pulse smooth without abandoning render-on-demand for the rest of the scene.

6.3 Object Markers & Styling Rules

Every catalogue object is drawn as a point whose colour is fixed by **category** and whose size and label are fixed by **zenith state**. The rules are strict and centralized in a `getPointStyle()` helper:

Object Class	Colour	Size	Label
Satellite (non-zenith)	#4fc3f7 cyan @ 0.2 α	2 px	Hidden
Satellite (in zenith)	#4fc3f7 cyan	7 px	Shown
ISS	#ffcc02 gold	10 px	Always
Planet	#ff8c69 salmon	8 px	Always

■ THE ISS NAME-MATCHER BUG

An early matcher tagged any object whose name merely *contained* "ISS" as the station — which wrongly caught SWISSCUBE, WEISS, BLITS and lit them gold. The fix is a single shared, word-boundary-anchored matcher: `/\bISS\b|ZARYA/i`, used identically by both the worker's categorizer and the ISS-promotion step so they can never disagree. Exactly one object is ever the ISS.

6.4 Orbital Trail Lines

Objects inside the Zenith Window (and the ISS at all times) grow a glowing orbital trail. A per-object ring buffer retains the last several positions; a `PolylineGlowMaterialProperty` draws them as a fading streak — gold for the ISS, cyan for satellites. When an object leaves the window its trail entity is removed and its history cleared. Planets never receive trails.

7. Complete Feature Catalogue

This section documents every user-facing feature in the application, the logic behind it, and the design decisions baked into each one.

7.1 Observer Location Picker

A glassmorphism panel, toggled from a TopBar button that displays the current observer label (e.g. ■ Chennai). It offers three ways to set location, and every path ultimately calls `setObserver({ latitude, longitude, altitudeM, label })` and closes the panel:

- **Use my location** — the browser Geolocation API with high-accuracy and a 10-second timeout, with graceful handling of permission-denied and unsupported cases.
- **City search** — the query is sent to the `/api/geocode` proxy (Nominatim), which returns the top matches as `{ label, latitude, longitude }` for click-to-select.
- **Manual coordinates** — two number inputs with range validation (lat ± 90 , lng ± 180).

When the observer changes, two existing store subscriptions react automatically: the cone and observer marker re-render immediately, and the next pipeline tick re-derives every topocentric angle and the zenith set. The globe also performs a cinematic fly-to (§7.8).

■ GEOLOCATION REQUIRES HTTPS OR LOCALHOST

The browser Geolocation API only functions over `https` or `localhost`. If the app is ever served over a plain `http` LAN IP for a live demo, "Use my location" silently fails — so a fallback message and a sensible default observer are needed for that scenario.

7.2 Pass Predictions

For any selected satellite or the ISS, Zenith predicts upcoming visible passes over the next 24 hours. `lib/passPredictions.ts` steps forward in 10-second increments, propagating the TLE at each step and tracking the topocentric altitude. A pass begins when altitude rises above 10° , ends when it falls back below, and its peak altitude is the highest value reached in between. Each `PassEvent` records rise time, set time, peak altitude, peak azimuth, and duration.

Because a 24-hour search at 10-second resolution is roughly 8,640 propagations per object, the work runs in the Web Worker via the `PREDICT_PASSES` message, which posts back `PASS_PREDICTIONS`. The `PassPredictionPanel` shows a spinner while computing, then renders each pass as a card; if there are none it states so plainly. Passes are shown for satellites and the ISS, but hidden for planets, whose "passes" are not meaningful in the same sense.

7.3 Object Detail Panel

Clicking any object slides in a detail panel from the right. It shows the object name, a colour-coded category badge, and live-updating telemetry refreshed on a one-second interval: current altitude, azimuth, latitude, longitude, and height. Context-specific lines adapt to the object type:

- **Satellites** show the TLE epoch age in days — how stale the orbital data is.
- **The ISS** is labelled "Live position via OpenNotify."
- **Planets** are labelled "Ephemeris via NASA Horizons."

- An object inside the window earns a glowing, pulsing "IN ZENITH WINDOW" badge.

The `PassPredictionPanel` is embedded inside this panel for satellites and the ISS. Clicking empty globe space, the `X` button, or pressing `Escape` deselects and slides the panel away.

7.4 Time Machine

A slider with four snap points — `Now`, `+1h`, `+6h`, `+24h` — lets the user fast-forward the simulation. The selected offset is stored as `offsetHours`; the refresh loop adds it (in milliseconds) to the current timestamp before propagation, so every satellite, the ISS, and the planets jump to where they will be at the offset time, and the Zenith Window recomputes accordingly. A small pill shows the active offset whenever it is non-zero.

7.5 Zenith Radar Minimap

A bottom-left polar planisphere of the local sky. It plots **only** the objects currently in the Zenith Window — deliberately filtered to stay legible rather than mirroring the full swarm. Cardinal markers (N/E/S/W) ring the edge; the 75°–90° zenith band is shaded as a concentric ring at the centre; and a sonar-style pulse ring expands and fades on a loop. The ISS is drawn in magenta, other zenith objects in cyan. The pulse animation is driven by `requestAnimationFrame` rather than a timer, so it pauses automatically when the tab is backgrounded.

7.6 Third-Person Satellite Chase View

Clicking a satellite or the ISS enters a cinematic chase mode. A 3D satellite model (a custom GLB from Sketchfab, shared by all satellites; the ISS rendered slightly larger and gold) is placed at the object's live position, oriented along its direction of travel by computing the delta between successive positions. A `scene.preUpdate` listener runs every frame: it updates the model's position from the store and re-aims the camera using `lookAt` with a `HeadingPitchRange` offset, sitting above and behind the satellite with the camera distance scaled to the object's orbital altitude. The tracked object's flat point marker is hidden so the dot doesn't overlap the model.

A pulsing "● TRACKING" pill plus an "✕ Exit Tracking" button appear in the detail panel; `Escape` also exits. On exit the camera lock is released with `lookAtTransform(Matrix4.IDENTITY)` — which must be called *before* the fly-back — and the camera flies smoothly back to the observer view.

THE EXIT-CAMERA GOTCHA

Calling `flyTo` while the camera is still locked in a `lookAt` reference frame leaves it stuck. The fix is ordering: release the lock with `lookAtTransform(Cesium.Matrix4.IDENTITY)` first, then fly back to the observer.

7.7 Solar-System View

Zooming far enough out — past a camera-altitude threshold — transitions the scene into a full solar system with the Sun at the origin and all eight planets orbiting it as 3D GLB models. A dedicated ■ `Solar System` button flies the camera out to trigger the same threshold, and toggles to ■ `Back to Earth` to return. The implementation:

- **Scaled distances.** Real astronomical units are scaled down by a constant so the whole system fits in a navigable Cesium scene; each body has its own orbital radius, model scale, and rotation period defined in `lib/solarSystem.ts`.
- **Live orbital motion.** A `preUpdate` animation advances each planet around its orbit (faster real-time-to-year mapping so motion is visible) and spins each body on its axis.
- **Orbit rings.** Each planet's orbital path is drawn as a faint 360-point circle in its own colour.
- **Earth hidden.** The built-in terrain globe is hidden in this mode (it would appear as a meaningless dot at the Sun's position), and the satellite collections are hidden too — the whole thing is an additive overlay that cleanly tears down on return.
- **Clickable planets.** Selecting a planet opens the detail panel and reuses the chase-camera machinery, scaled up to planetary distances. The Sun is selectable but stays fixed at the centre.

7.8 Cinematic Camera Choreography

Location changes and app load are staged as camera flights rather than instant jumps. On load, the camera sets a high orbital view of the default observer (Chennai) and then flies down to a 2,500 km vantage at a 45° downward pitch over several seconds with cubic easing. Every observer change — from search, geolocation, or manual entry — triggers the same smooth fly-to so the globe appears to rotate to the new location and reveal its cone. The default observer throughout is **Chennai (12.9716°N, 80.2437°E)**, near VIT Vellore.

7.9 Constellation View

A ♦ Constellations toggle reveals the constellations visible from the observer's location, drawn directly onto the celestial sphere around the scene. The feature works as follows:

- **Star catalogue.** `lib/constellationData.ts` holds the prominent IAU constellations with their bright stars (RA/Dec, magnitude) and the stick-figure line connections that form each pattern.
- **Celestial-sphere projection.** `raDecToCartesian()` maps each star's right ascension and declination onto a sphere billions of metres in radius, so the stars sit as a sky backdrop around the Earth scene.
- **Overhead detection.** The observer's zenith RA/Dec is computed from their longitude and the current Local Sidereal Time. The constellation closest to that zenith point is treated as **overhead**; if none is within the boundary threshold, the **nearest** visible constellation is highlighted instead — satisfying the requirement that something is always shown.
- **Highlighting.** The overhead/nearest constellation's lines glow cyan and its stars enlarge; all others render faintly. A toast announces which constellation is overhead.
- **Immersive 3D view.** Clicking a constellation flies the camera to face its centre, intensifies its highlight, fades the rest, and adds a scattered background starfield for depth. A `ConstellationPanel` shows the name, abbreviation, an OVERHEAD/NEAREST badge, a one-line mythology note, and the named bright stars with their magnitudes.
- **Isolation.** Constellations use their own dedicated primitive collections, entirely separate from the satellite collections, so the two never interfere.

A PERFORMANCE-DRIVEN ROUTING DECISION

To keep the heavy constellation rendering from competing with the main globe, the feature was ultimately isolated onto its own `/constellations` route (with observer coordinates passed through URL search params), running its own render-on-demand Cesium instance. This protects the main page's frame rate — an instance of the broader pattern of moving expensive Cesium features onto separate routes rather than burdening the primary scene.

8. UI / UX Design System

Because the hackathon weights the interface heavily, the visual language is treated as a first-class engineering concern. The system is a **dark-space glassmorphism** aesthetic: a deep obsidian void, a photoreal Earth as the primary light source, and frosted-glass panels with glowing cyan edges floating above it.

8.1 Colour Language

Role	Hex	Applied To
Cyan (primary)	#06b6d4 / #4fc3f7	Brand accent, satellites, cone, borders, glow
Gold	#ffcc02	The ISS — everywhere, consistently
Salmon	#ff8c69	Planets
Magenta	#ff69b4	ISS marker on the radar minimap
Slate	#64748b / #94a3b8	Secondary / telemetry text
Obsidian	#0a0e1a	The void background

8.2 Glassmorphism Recipe

Every panel shares one container recipe, giving the interface a unified, high-tech HUD feel while letting the stars and Earth bleed softly through:

```
bg-black/30 backdrop-blur-md border border-cyan-500/20 rounded-xl

Object/satellite names → text-white font-semibold
Telemetry (Alt/Az/range) → text-slate-400 text-xs font-mono
Category badges → tinted pill (cyan / yellow / orange) with matching border
```

8.3 Typography

Space Grotesk carries display headings and object names — geometric, modern, a little futuristic. **Space Mono** handles all numeric telemetry, labels, and badges, reinforcing the mission-control, data-driven character and giving orbital figures a precise, instrument-readout quality.

8.4 Motion & Animation Inventory

Animation is used purposefully to make the interface feel alive without becoming noisy. The complete inventory:

Animation	Behaviour
Cosmic fade-in	Whole UI scales/fades in gently on load
Zenith Score ring	SVG stroke arc animates to the score over ~1 s
Score waveform	Row of bars oscillating on staggered delays (live-signal feel)
Zenith list items	New entries fade + slide in from the right

Animation	Behaviour
Cone breathing	Cone opacity + outline pulse on a sine wave each frame
Radar sonar pulse	Expanding, fading ring on the minimap (rAF-driven)
IN ZENITH badge	Glow pulse on the in-window badge
Panel slide	Detail / observer panels slide + ease in and out
Camera flights	Eased fly-to on load, location change, and tracking exit
Planet orbits	Continuous orbital + axial motion in solar-system view

8.5 The Decluttering Principle

A central UX insight shaped the look: rendering all ~750 objects as bright, equal dots produced an opaque swarm that buried the Earth. The resolution was a strict visual hierarchy — non-zenith satellites shrink to faint 2 px cyan specks, while only the handful of in-window objects (plus the ISS and planets) are enlarged, brightened, labelled, and trailed. The Earth stays beautiful and the eye is guided straight to what matters: the objects overhead.

9. Server-Side API Routes

All four external integrations are fronted by Next.js route handlers under `/app/api/`. Each owns its own caching window and graceful-failure behaviour.

Route	Upstream	Cache	Failure Behaviour
<code>/api/tle</code>	CelesTrak GP	2 h (localStorage client cache)	502 when all feeds fail with no cache; loop retries
<code>/api/iss</code>	OpenNotify	3 s (route revalidate)	502 → ISS falls back to its SGP4 position
<code>/api/planets</code>	JPL Horizons	60 s (route revalidate)	Serves last-known cache; 503 if no cache → planets skipped this tick
<code>/api/geocode</code>	OSM Nominatim	1 h	400 on empty query; 502 on upstream failure

9.1 ISS — Dual-Source Promotion

The ISS is handled with special care. After the satellite array is built from TLEs, the object matching the anchored `ISS_NAME_PATTERN` has its category overridden to `iss`. The pipeline then fetches `/api/iss` and replaces that object's position with the live OpenNotify coordinates — more accurate for the ISS than SGP4, since OpenNotify is fed directly by NASA. Crucially, this *mutates the existing object in place* rather than creating a second one, and if the fetch fails it keeps the SGP4 position and warns — so there is always exactly one ISS and it is never dropped.

9.2 Planets — Horizons Ephemeris

`/api/planets` queries JPL Horizons for the geocentric right-ascension, declination, and range of the visible planets, converts those to geodetic positions through the same helpers the satellite pipeline uses, and returns them as `CelestialObjects` with category `planet`. The pipeline computes each planet's topocentric Alt/Az against the observer and its zenith flag, then appends them before the store upsert. A module-level last-known cache makes the route resilient to Horizons outages.

10. Type System

types/celestial.ts defines the shared vocabulary that keeps the pipeline, store, and UI in agreement. A single unified CelestialObject type represents satellites, the ISS, and planets alike, so the rendering and panel code can treat them uniformly while branching on category where needed.

Type	Shape / Purpose
CelestialObject	Unified object: id, name, category, geoPosition, topocentricPosition, inZenithWindow, TLE lines (satellites)
TopocentricPosition	{ altitude, azimuth, rangeKm } — the observer-relative look angles
GeoPosition	{ latitude, longitude, heightKm } — WGS-84, for Cesium placement
ObserverLocation	{ latitude, longitude, altitudeM, label } — the viewer's location
PassEvent	{ riseTime, setTime, maxAltitude, maxAzimuth, durationSeconds }
ConstellationData	{ id, name, abbreviation, stars[], lines[], centerRa, centerDec, mythology }

Categories are a small fixed union — 'satellite' | 'iss' | 'planet' (extended with solar-system and constellation identifiers for those features) — which makes the strict colour and styling rules in the renderer exhaustive and type-checked.

11. Performance Engineering

At one point the application became badly laggy. Rather than guess, the team ran a structured diagnosis of the render path, found the true bottleneck, and rebuilt the CesiumJS layer around batched primitives and render-on-demand. The result was a transformation from continuous per-frame work to an app that does a bounded amount of work once per data tick and otherwise sits idle. This section is the full account.

11.1 The Diagnosis

The lag was traced to the **render loop**, not data processing. The ranked causes:

Rank	Cause	Why It Hurt
#1	~750 time-dynamic entities	Each object was a Cesium Entity with a <code>SampledPositionProperty</code> ; with the clock animating, Cesium re-interpolated all ~750 positions every single frame at 60 fps.
#2	~750 label graphics	Every object carried a label even when hidden; labels are the most expensive entity graphic and Cesium still manages a text texture per entity.
#3	EntityCollection, not primitives	Individual entities meant no draw-call batching — 750 separate managed objects instead of one.
#4	Radar timer	A 50 ms <code>setInterval</code> forced ~20 React re-renders per second, independent of the data ticks.
#5	Trail array churn	Trail polylines lived on the entity path and reassigned a fresh positions array every tick, forcing geometry rebuilds.

Notably, three things were confirmed *not* to be the problem: SGP4 (already off-thread in the worker), the marker-sync frequency (throttled to one call per animation frame and only triggered every 10 s), and the globe component itself (it does not React-re-render on store updates).

11.2 The Six Fixes

Fix 1 — Batched points → `PointPrimitiveCollection`

All ~750 per-object entities were replaced by a single `PointPrimitiveCollection` created once. A `Map<id, PointPrimitive>` cache lets each tick update an existing primitive's position/colour/size or add/remove as needed. ~750 managed objects collapse into one batched draw call — the dominant win.

Fix 2 — Labels → `LabelCollection`, zenith/ISS/planet only

A single `LabelCollection` carries labels, and a label is created *only* for in-window objects, the ISS, and planets. The ~740 non-zenith satellites get no label object at all — eliminating the most expensive graphic for the vast majority of the catalogue.

Fix 3 — Remove `SampledPositionProperty` entirely

All time-sampled interpolation was deleted. Positions are now plain static `Cartesian3` values written once per 10-second tick, so Cesium no longer re-evaluates 750 dynamic properties every frame.

Fix 4 — Trails → `PolylineCollection`

Trail lines moved to a single batched `PolylineCollection` with their own caches, updated in place rather than as churning individual entities.

Fix 5 — Clock off + render-on-demand

`clock.shouldAnimate = false, requestRenderMode = true,` and `maximumRenderTimeChange = Infinity` switch Cesium from a continuous 60 fps loop to redrawing only when something actually changes. Explicit `requestRender()` calls are placed at the end of marker sync, cone render, and observer-marker render. Idle GPU/CPU between ticks drops to nearly zero.

Fix 6 — Radar timer → `requestAnimationFrame`

The 50 ms interval was replaced by a single `RAF` loop computing the pulse from elapsed time with one batched state write per frame — paced to the display and auto-paused when the tab is hidden, removing the steady 20-renders-per-second tax.

NET EFFECT

The continuous per-frame work — 750 interpolations + 750 labels + a 60 fps redraw — was eliminated. The app now performs a bounded burst of work once per 10-second tick and otherwise idles, with the radar pulse as the only lightweight ongoing animation. The swarm became a clean, smooth, glanceable scene.

12. Engineering Lessons & Key Principles

Six days of building produced a set of durable lessons — several of them bugs that cost real time and are now permanent guardrails in the project's context.

12.1 Technical Gotchas (Hard-Won)

Lesson	Detail
Radians vs degrees	ecfToLookAngles returns elevation in radians; must $\times 180/\pi$ before comparing to the 75–90° window. The single most consequential bug.
ISS name matching	Use a word-boundary regex (<code>(\bISS\b ZARYA)</code>), shared by worker and promotion, or substrings like SWISSCUBE get mis-tagged as the station.
CelesTrak rate limits	Use the stable GP endpoint with a 2 h localStorage cache; bursts of reloads will get the IP throttled.
Geolocation needs HTTPS	Browser geolocation only works over https/localhost; LAN http demos need a fallback message and a default observer.
Camera lock on exit	Release <code>lookAtTransform(IDENTITY)</code> before <code>flyTo</code> , or the camera stays stuck after chase mode.
Cesium clock = per-frame cost	<code>shouldAnimate + SampledPositionProperty</code> re-evaluates every entity every frame; static positions + render-on-demand are far cheaper.

12.2 Workflow Principles

The project was implemented through a disciplined human-plus-AI workflow that itself became a key success factor:

- **Architecture vs implementation split.** High-level design and decision-making were kept separate from code generation; implementation was driven by detailed, copy-pasteable, tightly-scoped prompts.
- **Incremental passes.** Large context dumps caused hallucinated file structures, skipped sections, and lost variable names. The fix was to split every change into small, logically independent passes — data/utilities first, then scaffolding, then camera, then UI wiring, then type-cleanup.
- **Separate Cesium and React/CSS changes.** WebGL rendering logic and React/CSS work were never mixed in a single change set, avoiding a whole class of conflicts.
- **Protect working code.** Every change began with explicit file-read instructions and a DO-NOT-TOUCH list shielding known-good code, with scope tightly defined per change.
- **Smoke-test feedback loop.** Detailed per-change status reports after each run made it possible to track precisely what was applied versus still pending.
- **Persistent project context.** A living context file (CLAUDE.md) carried the stack, conventions, structure, and hard-won lessons across sessions so they never had to be rediscovered.
- **Isolate heavy features onto routes.** Expensive Cesium features (e.g. constellations) were moved onto their own render-on-demand routes rather than burdening the main globe.

13. Build Timeline

The project was built as a multi-day sprint, each day a self-contained, testable milestone.

Phase	Theme	Delivered
Day 1	Foundation	Next.js scaffold, CesiumJS globe, Zustand store, dev seed
Day 2	Data pipeline	CelesTrak TLEs, SGP4 propagation, Alt/Az, the radians fix
Day 3	Zenith Window	Radar cone + real-time markers + the 10 s refresh loop
Day 4	Live integrations	ISS via OpenNotify, planets via Horizons, ISS name-matcher fix
Day 5	UI panels	Observer picker, pass predictions, object detail panel
Day 6	Polish	Glassmorphism, Zenith Score, Time Machine, radar, animations
Beyond	Advanced features	Performance rebuild, 3D chase camera, solar system, constellations

13.1 Current State

The application is feature-rich and demo-ready: ~750 satellites tracked live; a breathing Zenith cone with topocentric markers; the live ISS and the visible planets; 24-hour pass predictions; a third-person 3D satellite chase camera with a custom GLB model; a fully orbiting solar-system view with planet models and orbit rings; an overhead/nearest constellation view with immersive 3D mode; a complete dark-space glassmorphism UI with an animated Zenith Score, sonar radar, and Time Machine; and a rebuilt, render-on-demand performance pipeline keeping it all smooth.

Project Zenith — The Celestial Eye

Team Cipher · AstralWeb Innovate · Aaruush '26

"What is above me right now?" — answered beautifully, in real time.